



Actividad de recuperación día 7

ARREGLOS, MÉTODOS DE ARREGLO Y FORMATO DE MONEDA

Fecha	Domingo 27 de septiembre de 2026
Aprendiz	Sneider Alvarez Santos
Ficha	3428392
Documento	1016717046
Programa	Desarrollo web — SENA
Proyecto	La Carambola Dorada — sistema de gestión de salones de billar
Publicación	Servidor local XAMPP — http://localhost/plan-mejoramiento-desarrollo-web/

Bitácora del día: qué hice

- 1 Creé `js/datos-prueba.js` con **16 productos y 8 pedidos**, usando la misma forma que tendrán sus tablas en la base de datos.
- 2 Resolví en `js/ejercicios.js` los cuatro cálculos con métodos de arreglo encadenados, **sin una sola línea de for**.
- 3 Formateé los valores monetarios con `Intl.NumberFormat` en pesos colombianos.
- 4 Escribí esta bitácora con la diferencia entre `map` y `forEach` y un ejemplo propio del salón.

Qué error encontré y cómo lo resolví

Una comprobación que siempre daó falso, y dice que la culpa es de los datos.

Quería confirmar que el producto más caro no tuviera empate con otro. Escribí:

```
console.log("maximo unico: " + cuantos === 1);
```

La consola respondió `false`. Persigué el error en el algoritmo del `reduce` y casi dejo el cálculo. El problema no estaba en los datos: el `+` **tiene más precedencia que el `===`**, así que JavaScript no comparaba un número con `1`, sino la cadena ya pegada `"maximo unico: 1"` contra el número `1`. Una cadena nunca es igual a un número, así que el resultado era `false` siempre, ganara o no hubiera empate.

```
typeof ("n=" + 1) === "string" // true -> el + ya se aplico  
typeof "n=" + 1 === "string" // false -> el + se comio la comparacion
```

La solución fue dos cosas: poner el paréntesis `("maximo unico: " + cuantos) === 1`, y dejar de comparar números ya impresos. Ahora la comprobación recibe y devuelve el número pelado, y el texto lo arma la línea que muestra el resultado.

Lo que me llevo: una prueba que siempre falla es peor que no tenerla, porque parece un error del código que uno acaba de escribir y hace perder el rato en el sitio equivocado. Si una aserción falla, primero hay que confirmar que está mirando lo que uno cree que está mirando.

Este día me quedó la lección de no escribir a ojo. Los cuatro resultados que aparecen más abajo no los calculé de memoria: los ejecuté en el navegador y los copié de allí. Cuando lo hice así, un promedio que yo había estimado en \$ 20.656,25 salió en realidad \$ 19.531,25.

Punto 1 — Los datos de prueba

El archivo es `js/datos-prueba.js`. Cada objeto tiene la forma que tendrá su tabla en la base de datos: los mismos campos y los mismos nombres que ya usa `productos.html`, y las mismas cuatro categorías del formulario.

CAMPO	TIPO	QUÉ ES
<code>codigo</code>	texto	El código único, como <code>TAC-001</code>
<code>nombre</code>	texto	Lo que se ve en la etiqueta
<code>categoria</code>	texto	Una de las cuatro: implemento de juego, consumible, repuesto o mesa de billar
<code>condicion</code>	texto	Nuevo, semi-nuevo o en reparación
<code>existencias</code>	número	Cuántas hay en el salón
<code>valor_alquiler</code>	número	Pesostegeros, sin signo ni separador

El detalle que hace que sirva: `valor_alquiler` se guarda como número pelado, `12000`, nunca como `"$ 12.000"`. Un número con signo de pesos dentro no se puede sumar: `"$ 12.000" + "$ 8.000"` produce una cadena inútil. El formato se le pone al mostrar, nunca al guardar. Esa es la diferencia entre un dato que después se puede consultar y uno que no.

Los 16 productos

Seis son los que ya estaban en `productos.html` y los mantuve tal cual, para que el arreglo no se contradiga con la página. Los otros diez se inventaron con el mismo `formato XXX-000`:

CÓDIGO	PRODUCTO	CATEGORÍA	CONDICIÓN	EXISTENCIAS	ALQUILER
TAC-001	Taco de pool 1,45 m	Implemento de juego	Nuevo	8	\$ 12.000
BOL-004	Juego de bolas de billar	Implemento de juego	Nuevo	3	\$ 45.000
TRI-002	Triángulo para bolas	Implemento de juego	Nuevo	5	\$ 8.000
EXT-005	Extensión de taco	Implemento de juego	En reparación	1	\$ 10.000
PUN-006	Puntería profesional	Implemento de juego	Nuevo	4	\$ 18.000
SOP-016	Soporete para taco	Implemento de juego	Nuevo	15	\$ 5.000

CÓDIGO	PRODUCTO	CATEGORÍA	CONDICIÓN	EXISTENCIAS	ALQUILER
TIZ-010	Tiza de billar	Consumible	Nuevo	24	\$ 2.000
GUA-003	Guantes de fibra	Consumible	Semi-nuevo	6	\$ 15.000
GUA-007	Guantes de cuero sintético	Consumible	Semi-nuevo	7	\$ 22.000
AGT-014	Agente de limpieza para mesas	Consumible	Nuevo	12	\$ 7.500
TIZ-015	Tiza de Tournament	Consumible	Nuevo	20	\$ 4.500
REP-011	Repuesto de punta de taco	Repuesto	Nuevo	18	\$ 3.500
REP-012	Banda de goma para pocket	Repuesto	Nuevo	9	\$ 6.000
CON-013	Conector de extensión	Repuesto	En reparación	2	\$ 9.000
MES-008	Mesa de billar 8 pies	Mesa de billar	Nuevo	2	\$ 60.000
MES-009	Mesa de billar 9 pies	Mesa de billar	Nuevo	1	\$ 85.000

Los 8 pedidos

Cada pedido trae su cliente, su fecha, su estado y la lista de items con el código del producto y la cantidad. El `total` está escrito como número, igual que el precio del producto, y se comprobó que los ocho cuadran con la suma de sus items.

```
{
  id: "PED-002", cliente: "Federación Antioqueña de Pool",
  fecha: "2026-09-28", estado: "Confirmado",
  items: [
    { codigo: "MES-009", cantidad: 1 },
    { codigo: "BOL-004", cantidad: 2 },
    { codigo: "PUN-006", cantidad: 4 }
  ],
  total: 247000
}
```

Ojo con esto: `PED-002` pide una mesa de 9 pies (`85.000`), dos juegos de bolas (`2 × 45.000`) y cuatro punterías (`4 × 18.000`). Suman `247.000`, que es el `total` que dice el pedido.

Punto 2 — Los cuatro cálculos con métodos de arreglo

Todo está en `js/ejercicios.js`. No hay un solo `for`, ni `for...of`, ni `while`: cada cálculo es un `reduce`, un `filter` o un `map`.

1. El producto más caro

Un `reduce` que lleva guardando el ganador. El ternario decide si el producto que está pasando ahora supera al que ya tenía.

```
const productoMasCaro = productos.reduce((masCaro, producto) =>
  producto.valor_alquiler > masCaro.valor_alquiler ? producto : masCaro
);
```

CÓDIGO	PRODUCTO	CATEGORÍA	ALQUILER
MES-009	Mesa de billar 9 pies	Mesa de billar	\$ 85.000

Sin empate: el siguiente en la lista es la mesa de 8 pies, a \$ 60.000. Eso lo comprobé el script, no lo supuse.

2. Total de unidades por categoría

El `reduce` arranca con un objeto vacío `{}` como acumulador y va creando cada categoría la primera vez que aparece.

```
const unidadesPorCategoría = productos.reduce((acumulado, producto) => {
  acumulado[producto.categoría] =
    (acumulado[producto.categoría] ?? 0) + producto.existencias;
  return acumulado;
}, {});
```

El `?? 0` no es un detalle de estilo. Si se escribe `acumulado[producto.categoría] + producto.existencias` sin el `?? 0`, la primera vez que pasa un producto de cada categoría la categoría todavía no existe, y `undefined + 8` da `NaN`. Lo probé y las cuatro categorías quedaron en `NaN`: ni 36, ni 69, ni nada. Un solo detalle de dos caracteres decide si el cálculo sirve.

CATEGORÍA	UNIDADES
Implemento de juego	36
Consumible	69
Repuesto	29
Mesa de billar	3

3. Productos con existencias menores a cinco

Aquí sí se encadenan dos métodos: `filter` deja pasar los que cumplen, y `sort` los ordena de menos a más para que el más urgente quede primero.

```
const stockBajo = productos
  .filter(producto => producto.existencias < 5)
  .sort((a, b) => a.existencias - b.existencias);
```

CÓDIGO	PRODUCTO	CONDICIÓN	EXISTENCIAS	ALQUILER
EXT-005	Extensión de taco	En reparación	1	\$ 10.000
MES-009	Mesa de billar 9 pies	Nuevo	1	\$ 85.000
CON-013	Conector de extensión	En reparación	2	\$ 9.000
MES-008	Mesa de billar 8 pies	Nuevo	2	\$ 60.000
BOL-004	Juego de bolas de billar	Nuevo	3	\$ 45.000
PUN-006	Puntería profesional	Nuevo	4	\$ 18.000

Seis de dieciséis, y son justo las dos mesas, las dos piezas en reparación y las dos que tienen menos de cinco. Es el orden en que habría que llamar al proveedor.

4. Promedio de precio de alquiler

Tres pasos encadenados: `map` extrae los precios, `reduce` los suma, y el resultado se divide entre la cantidad de productos.

```
const promedioPrecio = productos
  .map(producto => producto.valor_alquiler)
  .reduce((suma, valor) => suma + valor, 0) / productos.length;
```

MEDIDA	VALOR
Suma de los 16 precios	312500
Códigos	16
Promedio	\$ 19.531,25

Punto 3 — La moneda con Intl.NumberFormat

No se escribe el signo ni el punto a mano. `Intl.NumberFormat` sabe las reglas del país: dónde va el símbolo, dónde los puntos, cómo se separan los miles.

```
const pesos = new Intl.NumberFormat("es-CO", {
  style: "currency",
  currency: "COP",
  maximumFractionDigits: 0
});
```

Con eso, `pesos.format(85000)` devuelve `"$ 85.000"` y `pesos.format(12000)` devuelve `"$ 12.000"`, sin que yo escriba un solo punto en el código.

Por qué hacen falta dos formateadores

El peso colombiano no tiene subdivision en uso, así que los precios van sin decimales. Pero el **promedio** del punto 4 sí cae en centavos, y ahí un solo formateador mentiría:

CON <code>MAXIMUMFRACTIONDIGITS: 0</code>	SALIDA
Promedio real: <code>19531.25</code>	\$ 19.531,25
El mismo número con el formateador de precios	\$ 19.531

Esos dos números no son lo mismo: falta el medio. Así que el promedio usa un segundo formateador, que obliga a mostrar los centavos.

```
const pesosConCentavos = new Intl.NumberFormat("es-CO", {
  style: "currency",
  currency: "COP",
  minimumFractionDigits: 2,
  maximumFractionDigits: 2
});
```

Un detalle que se nota al revisar la salida: `Intl` separa el símbolo del número con un **espacio duro**, no con un espacio normal. Si algún día busca `"$ 85.000"` con un espacio corriente, no lo va a encontrar, porque el carácter que hay ahí es `U+00A0`. Por eso el formateo se deja en manos

de `Intl` y no se arma a mano: es más corto y además sale bien en los 16 casos, no solo en el que yo probé.

Punto 4 — Cuándo usar `map` y cuándo `forEach`

Los dos recorren el arreglo igual y los dos se ven casi iguales en el papel. La diferencia es **qué hacen con lo que les pides**.

El ejemplo propio, con los datos del salón

Quiero una lista con el nombre de los productos que se pueden prestar, para rotular los cajones:

```
const nombresAlquilables = productos
  .map(producto => producto.nombre);
console.table(nombresAlquilables);
```

Eso devuelve **un arreglo nuevo con los 16 nombres**. El arreglo `productos` sigue intacto, con sus categorías, sus existencias y su condición. Y el resultado se puede seguir usando, guardar o pasar a otra parte.

Ahora el mismo recorrido, pero solo para mostrarlo en la consola:

```
productos.forEach(producto => {
  console.log(` ${producto.codigo}: ${producto.existencias} en existencia`);
});
```

Aquí no quiero una lista nueva: quiero que pase algo. `forEach` no devuelve nada, devuelve `undefined`. Lo único que hace es ejecutar la función una vez por cada elemento.

Por eso está mal escribir `forEach` donde va `map`: si después quiero usar el resultado, me encuentro con `undefined` y un error del tipo `map is not a function`, cuando el problema no era el método de recorrido sino que ese método nunca devolvió nada. Y al revés: poner `map` solo para imprimir es gastar un arreglo entero que se tira a la basura en el siguiente `console.log`.

Regla corta: si al final del recorrido quieres un valor, usa `map`, `filter` o `reduce`. Si solo quieres que algo ocurra —imprimir, sumar a otra variable, dibujar en la página—, `forEach`.

Qué recibe y qué devuelve cada método

Todos estos están en el mismo archivo, así que la tabla también:

MÉTODO	QUÉ RECIBE	QUÉ DEVUELVE
<code>map</code>	Una función que recibe el elemento, el índice y el arreglo	Un arreglo nuevo con el mismo largo, un resultado por cada elemento
<code>filter</code>	Una función que recibe el elemento, el índice y el arreglo	Un arreglo nuevo solo con los elementos que devolvieron <code>true</code> ; puede quedar corto o vacío
<code>reduce</code>	Una función que recibe el acumulador, el elemento, el índice y el arreglo, más el valor inicial	Un solo valor: el acumulador después el último elemento

MÉTODO	QUÉ RECIBE	QUÉ DEVUELVE
find	Una función que recibe el elemento, el índice y el arreglo	El primer elemento que cumple, o <code>undefined</code> si ninguno cumple
sort	Una función de comparación que recibe dos elementos	El mismo arreglo, ya ordenado. Ojo: lo modifica, no devuelve una copia
forEach	Una función que recibe el elemento, el índice y el arreglo	Nada: <code>undefined</code> . Solo produce efectos

La línea de abajo es la que más importa: casi todos reciben lo mismo, una función con el elemento, el índice y el arreglo. Lo que los separa es el valor de retorno, y ahí está la decisión de cuándo usar cada uno.

Criterios de evaluación que revise

CRITERIO	RESULTADO	CÓMO SE COMPROBÓ
Los cuatro cálculos con métodos de arreglo encadenados	OK	Cada cálculo es un <code>reduce</code> , <code>filter</code> o <code>map</code>
Cero ciclos <code>for</code> anidados	0	Búsqueda de <code>for</code> , <code>for...of</code> y <code>while</code> en el archivo
Al menos 15 productos y 8 pedidos	16 y 8	Conteo directo de los arreglos
Precios como número, no como texto	OK	Los 16 <code>valor_alquiler</code> se suman sin error
Los totales de los pedidos cuadran con sus items	8 de 8	Se recalcularon los 8 y se compararon
Moneda en pesos con <code>Intl.NumberFormat</code>	OK	Salida real de la consola, con los puntos de miles
Explico qué recibe y qué devuelve cada método	OK	La tabla del punto 4
Los dos scripts cargan sin error en el navegador	0 excepciones	Consola del navegador, 17 mensajes y ninguna excepción

Evidencia del día

Los dos archivos pedidos, y la consola del navegador con los resultados. La consola se capturó ejecutando los archivos en Edge, no con `console.log` copiado a mano.

- js/datos-prueba.js** — 16 productos y 8 pedidos, con la forma de sus futuras tablas.
- js/ejercicios.js** — los cuatro cálculos resueltos con métodos de arreglo y el formato de moneda.

• **Consola** Los cuatro cálculos del punto 2 y el formato de moneda del punto 3

Actividad día 7 – los cuatro cálculos con métodos de arreglo

16 productos y 8 pedidos cargados desde datos-prueba.js

1. Producto más caro

Código	Producto	Categoría	Condición	Existencias	Valor de alquiler
MES-009	Mesa de billar 9 pies	Mesa de billar	Nuevo	1	\$ 85.000

2. Total de unidades por categoría

Categoría	Unidades
Implemento de juego	36
Consumible	69
Repuesto	29
Mesa de billar	3

3. Productos con existencias menores a cinco

Código	Producto	Categoría	Condición	Existencias	Valor de alquiler
EXT-005	Extensión de taco	Implemento de juego	En reparación	1	\$ 10.000
MES-009	Mesa de billar 9 pies	Mesa de billar	Nuevo	1	\$ 85.000
CON-013	Conector de extensión	Repuesto	En reparación	2	\$ 9.000
MES-008	Mesa de billar 8 pies	Mesa de billar	Nuevo	2	\$ 60.000
BOL-004	Juego de bolas de billar	Implemento de juego	Nuevo	3	\$ 45.000
PUN-006	Puntería profesional	Implemento de juego	Nuevo	4	\$ 18.000

4. Promedio de precio de alquiler

\$ 19.531,25

Salida de la consola del navegador al cargar js/datos-prueba.js y js/ejercicios.js en este orden.

La consola con los cuatro cálculos. Se ve el producto más caro con `$ 85.000`, las cuatro categorías con sus unidades, los seis productos con existencias menores a cinco ordenados de menos a más, y el promedio en `$ 19.531,25`.

• **Consola** Los ocho pedidos de prueba del punto 1

Los ocho pedidos de prueba (punto 1)

Pedido	Cliente	Fecha	Estado	Ítems pedidos	Total
PED-001	Club de Billar La 8	2026-09-27	Confirmado	MES-008 x1, TAC-001 x4	\$ 108.000
PED-002	Federación Antioqueña de Pool	2026-09-28	Confirmado	MES-009 x1, BOL-004 x2, PUN-006 x4	\$ 247.000
PED-003	Cafeterías del Centro	2026-09-28	Pendiente	TIZ-010 x6, GUA-003 x2	\$ 42.000
PED-004	Universidad de Antioquia	2026-09-29	Entregado	SOP-016 x10, TRI-002 x2, REP-011 x4	\$ 80.000
PED-005	Club de Billar La 8	2026-09-29	Confirmado	MES-008 x1, MES-009 x1, BOL-004 x1	\$ 190.000
PED-006	Hotel Boutique del Río	2026-09-30	Pendiente	GUA-007 x4, EXT-005 x1	\$ 98.000
PED-007	Torneo Municipal de Pool	2026-10-01	Confirmado	MES-009 x1, MES-008 x1, PUN-006 x2, TIZ-015 x8	\$ 217.000
PED-008	Cafetería La Curva	2026-10-02	Entregado	TIZ-010 x4, AGT-014 x1, REP-012 x2	\$ 27.500

Salida de la consola del navegador al cargar js/datos-prueba.js y js/ejercicios.js en este orden.

Los ocho pedidos de prueba del punto 1, con el detalle de qué producto se pidió cuánto y el total ya formateado en pesos.

Cómo se tomó esta evidencia, para que quede claro: los dos scripts se cargaron en el navegador con dos etiquetas `<script>` seguidas, en el orden `datos-prueba.js` y luego `ejercicios.js`. Los 17 mensajes que emitieron se leyeron de la consola del navegador y se Guardaron textuales. La imagen es esa misma salida con formato para poder incluirla en el documento: **no es una fotografía del DevTools**, es la salida real, ya formateada para el documento. Si el instructor quiere ver el DevTools abierto, se abre el navegador y se corre en vivo.

